

CSF302

Software Engineering Department

Assignment 1

<Submitted by>

Student No:02240337

**Royal University of Bhutan, College of Science and
Technology**

QUESTIONS 1

Answer the following [1m]

- a. Write the formal definition of $f(n) = \Theta(g(n))$. Your answer must name every constant used and state the range of n over which the inequalities hold. (0.5)
- b. In the Word RAM model of computation, name the two quantities used to measure the cost of an algorithm. (0.5)

2. Consider the following function. Assume n is a power of 2 and $n \geq 2$. Use the counting convention stated in the Counting Convention section above.[3m]

```
{  
  int sum = 0, i, j;  
  for (i = 1; i <= n; i = i * 2)  
    for (j = 0; j < n; j++)  
      sum = sum + A[j];  
  return sum;  
}
```

- a. State, with justification, how many times the body of the outer loop executes, as a function of n . (0.5)
- b. Build the Cost / Repetitions / Total table for this function in the format used in Session 2. (1)
- c. From your table, derive the exact expression for $T(n)$. Simplify it fully. (1)
- d. State the tightest asymptotic bound for $T(n)$ in Θ notation, and justify it from your expression in
(c) by identifying the dominant term. (0.5)

Question 1

Ans: a) $\Theta(g(n))$:

$f(n) = \Theta(g(n))$ if and only if there exists positive constants c_1, c_2, n_0 such that

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0.$$

b) Word RAM cost measures: running time (number of primitive/word operations) and space (number of memory words or cells used)

Q2.

Ans: Outer loop count: i takes values $1, 2, 4, \dots, n$, which equal $2^0, 2^1, \dots$ up to 2 raised to the power $(\log \text{ base } 2 \text{ of } n)$. So it runs $(\log \text{ base } 2 \text{ of } n) + 1$ times.

$$\text{let } m = (\log \text{ base } 2 \text{ of } n) + 1.$$

b. ~~Word RAM cost measures:~~

Cost / Repetitions / Total table:

Statement	Cost	Repetitions	Total
int sum=0, i, j;	1	1	1
for (i=1; i<=n; i*=2)	2	$m+1$	$2m+2$
for (j=0; j<n; j++)	2	$m \text{ times } (n+1)$	$m(2n+2)$
sum = sum + A[j];	2	mn	$2mn$
return sum;	1	1	1

c. Exact $T(n)$:

$$T(n) = 1 + (2m+2) + m(2m+2) + 2mn + 1$$

$$= 4mn + 4m + 4$$

Substituting $m = (\log_2 n) + 1$:

$$T(n) = 4n \cdot [(\log_2 n) + 1] + 4 \cdot [(\log_2 n) + 1] + 4$$

$$= (4n + 4) \cdot (\log_2 n) + 4n + 8 //$$

D. Dominant term is $4n \text{ times } (\log_2 n)$, so $T(n) = \Theta(n \log n)$.

choose $c_1 = 4$

$c_2 = 8$

$n_0 = 2$

for $n \geq 2$.

$$4n(\log_2 n) \leq (4n + 4) \cdot (\log_2 n) + 4n + 8 \leq 8n \cdot (\log_2 n)$$

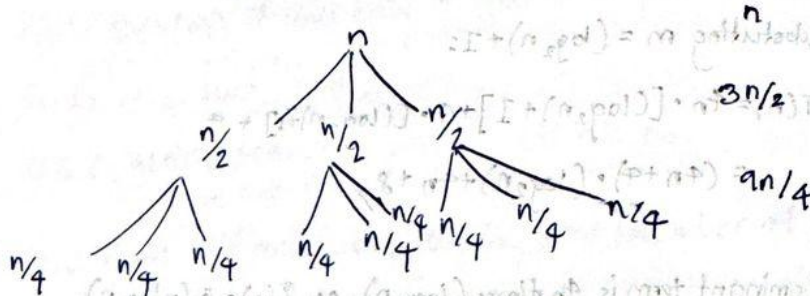
3. Analyze the recurrence $T(n) = 3T(n/2) + n$, with $T(1) = 1$, using the recursion tree method. (3m)
- a. Draw the top three levels of the recursion tree (levels 0, 1 and 2). Write the local cost inside each node and label each level with its total cost. (0.75)
 - b. (b) For a general level i , state: the number of nodes, the subproblem size, and the total cost of that level. Show how the level cost is obtained. (0.75)
 - c. (c) Give the height of the tree and the number of leaves, showing how you obtained each. (0.5)
 - d. (d) Sum the level costs. Identify the type of series you obtain, evaluate it, and state the resulting bound in Θ notation. (0.75)
 - e. (e) In one or two sentences, state which part of the tree the root, all levels equally, or the leaves dominates the total cost here. Contrast this with $T(n) = 2T(n/2) + n$ and $T(n) = 2T(n/2) + n^2$, both studied in class. (0.25)

Q5.

Ans: Recursion Tree for $T(n) = 3 \cdot T(n/2) + n$

a.

$$T(n) = 3 \cdot T(n/2) + n$$



b. General level i : 3ⁱ nodes each size of $n/2^i$.
each doing $n/2^i$ work.

$$\text{Level cost} = 3(n/2)$$

$$(n/2) \cdot 3 = \frac{n \cdot 3}{2}$$

c. Height & leaves: Tree ends when $n/2 = 1$.

$$\text{height} = \log_2 n$$

$$\text{leaves} = 3^{\log_2 n}$$

$$= n^{\log_2 3} //$$

d. Summing levels:

$\sum_{i=0}^{\log_2 n - 1} n(3/2)^i$ is an increasing geometric series, so its dominated by its last term, which is in order of the leaf count. gives a total of $\Theta(n^{\log_2 3})$ after evaluating.

e. Since the branching factor (3) exceeds the size reduction factor (2), the leaves dominate the total cost. Contrast: $T(n) = 2T(n/2) + n$ has $a=b=2$, so every level cost exactly $n \rightarrow$ all levels equal, giving $\Theta(n \log n)$. $T(n) = 2T(n/2) + n^2$ has $f(n)$ growing faster than the tree branches $\rightarrow$ root dominates, giving $\Theta(n^2)$.

4. A student submits the following proof for $T(n) = 4T(n/2) + n$, with $T(1) = 1$: (3m)

Guess. $T(n) \leq cn^2$

.

Inductive hypothesis. $T(k) \leq ck^2$ for all $k < n$.

Inductive step. $T(n) = 4T(n/2) + n \leq 4 \cdot c(n/2)^2 + n = cn^2 + n \leq cn^2$

,

since $+n$ is a lower-order term and is absorbed into the constant.

Therefore $T(n) = O(n^2$

).

- a. The stated bound $O(n^2)$ is in fact correct, but the proof is not valid.
- b. (a) Identify the exact line at which the argument fails and explain precisely why that line does not follow. (1)
- c. (b) Repair the proof by strengthening the hypothesis. State the strengthened claim in full. (0.5)
- d. (c) Carry out the inductive step for your strengthened claim in full, showing every algebraic line and the condition your new constant must satisfy. (1)
- e. (d) Verify the base case, and state explicit values of every constant and of n_0 for which your proof holds. (0.5)

Q4.

Ans: The falling line = $4c(n/2)^2 + n \leq cn^2$

This is false for every $c > 0$ and $n > 0$. We are claiming a positive number (n) doesn't increase cn^2 when added to it. The constant c is fixed once at the start of the proof; you can't invoke " n is lower-order" to make an additive term vanish against a fixed coefficient cn^2 . That only works when there is a term to subtract against.

b. Strengthened Hypothesis: $T(n) \leq cn^2 - dn$ with $c > 0, d > 0$.
 substrating a lower-order term gives the induction "device" to absorb the $+n$.

c. Inductive step.

$$T(n) = 4T(n/2) + n$$

$$\leq 4 \left[c \left(\frac{n}{2} \right)^2 - d \left(\frac{n}{2} \right) \right] + n$$

$$= 4 \left(\frac{cn^2}{4} - \frac{dn}{2} \right) + n$$

$$= cn^2 - 2dn + n$$

$$= cn^2 - (2d - 1)n$$

what we want:

$$cn^2 - (2d - 1)n \leq cn^2 - dn$$

$$(2d - 1)n \geq dn$$

$$d \geq 1 //$$

d. Base case: $n_0 = 1$

$$T(1) \leq c(1)^2 - d(1)$$

$$1 \leq c - d$$

$$\text{when } d = 1 \quad T(1) = 1 \leq 2 - 1$$

$$c = 2 \quad = 1$$

so the strengthened claim holds

$$T(n) \leq 2n^2 - n \text{ for } n \geq 1$$

$$\Rightarrow T(n) = O(n^2) //$$

5. The obvious way to find both the minimum and the maximum of an array of n elements is two separate scans, costing $2n - 2$ element comparisons. Design a divide-and-conquer algorithm MINMAX (A, lo, hi) that returns both values using strictly fewer comparisons, then verify your analysis experimentally.
- Write the pseudocode for MINMAX. Handle the base cases for one and two elements explicitly and use no more than two comparisons to combine the results of the two recursive calls. (1)
 - Let $C(n)$ be the number of element comparisons performed on an array of n elements. Derive the recurrence for $C(n)$ and state both base cases. (1)
 - Solve your recurrence using the iterative (unrolling) method. Show at least three expansions, give the general form after k expansions, substitute the value of k at which the base case is reached, and obtain a closed form. State the Θ bound. (1)
 - Implement MINMAX with a comparison counter. Run it on randomly generated arrays for $n = 8, 64, 512$ and 4096 , and present your results in this table
 - State whether measurement matches prediction. In one or two sentences, explain why the agreement here is exact rather than approximate. (1)

Q5.

Ans: a) Pseudocode:

MINMAX (A, lo, hi):

if $hi - lo == 0$:

return (A[lo], A[lo])

if $hi - lo == 1$:

if $A[lo] < A[hi]$:

return (A[lo], A[hi])

else:

return (A[hi], A[lo])

Comparison total

mid = $(lo + hi) // 2$

(min1, max1) = MINMAX (A, lo, mid)

(min2, max2) = MINMAX (A, mid+1, hi)

if min1 < min2: minVal = min1 else minVal = min2 # Comp

if max1 > max2: maxVal = max1 else maxVal = max2 # Comp

return (minVal, maxVal)

b) Recurrence: $C(n) = 2C(n/2) + 2$

with base case $C(2) = 1$

$C(1) = 0$

c) Solving by unrolling

$$C(n) = 2C\left(\frac{n}{2}\right) + 2$$

$$= 2\left[2C\left(\frac{n}{4}\right) + 2\right] + 2$$

$$C(n) = 2C\left(\frac{n}{2}\right) + 2$$

$$2\left[2C\left(\frac{n}{4}\right) + 2\right] + 2 = 4C\left(\frac{n}{4}\right) + 4 + 2$$

$$4\left[2C\left(\frac{n}{8}\right) + 2\right] + 4 + 2 = 8C\left(\frac{n}{8}\right) + 8 + 4 + 2$$

After k expansions:

$$C(n) = 2^k \cdot c\left(\frac{n}{2^k}\right) + 2^{k+1} - 2$$

We get this from:

$$C(n) = 2^k \cdot c\left(\frac{n}{2^k}\right) + (2^k + 2^{k-1} + \dots + 4 + 2)$$

Base case substitution:

$$\text{When } \frac{n}{2^k} = 2 \quad C(n) = 2^{\log_2 n - 1} \cdot C(2) + 2^{\log_2 n} - 2$$

$$k = \log_2 n - 1 \quad = \frac{n}{2} \cdot 1 + n - 2$$

$$= \frac{3n}{2} - 2$$

$$C(n) = \Theta(n) //$$

d)	n	Measured Comparisons	Predicted by closed form	$2n-2$ (two-sum method)
	8	10	10	14
	64	94	94	126
	512	766	766	1022
	4096	6142	6142	8190

e) Measurement matches prediction exactly. Comparisons are discrete, Everytime we run MINMAX on n elements, it does the exact number of comparisons the recurrence says. It does not depend on the actual values in the array. Also, since we only tested with n as power of 2, there were no floor or ceiling terms in the recursion. That means the closed form solution works out perfectly, not just approximately.

Code:

```
import random

def minmax_337(A337, lo337, hi337, counter337):
    if hi337 - lo337 == 0:
        return A337[lo337], A337[lo337]

    if hi337 - lo337 == 1:
        counter337[0] += 1
        if A337[lo337] < A337[hi337]:
            return A337[lo337], A337[hi337]
        else:
            return A337[hi337], A337[lo337]

    mid337 = (lo337 + hi337) // 2
    min1_337, max1_337 = minmax_337(A337, lo337, mid337, counter337)
    min2_337, max2_337 = minmax_337(A337, mid337 + 1, hi337, counter337)

    counter337[0] += 1
    minVal337 = min1_337 if min1_337 < min2_337 else min2_337

    counter337[0] += 1
    maxVal337 = max1_337 if max1_337 > max2_337 else max2_337

    return minVal337, maxVal337

def closed_form_337(n337):
    return (3 * n337) // 2 - 2

if __name__ == "__main__":
    sizes337 = [8, 64, 512, 4096]
    for n337 in sizes337:
        arr337 = random.sample(range(-1000000, 1000000), n337)
        counter337 = [0]
        result_min337, result_max337 = minmax_337(arr337, 0, n337 - 1, counter337)
        assert result_min337 == min(arr337)
        assert result_max337 == max(arr337)
        print(n337, counter337[0], closed_form_337(n337), 2*n337 - 2)
```